Linear system with Gauss-Seidel method :

Full name: khadija boumakla

Major : M1 data science and ai

Group : 2

Academic Year: 2026 – 2027

Supervised by A.EL IDRISSI

Gauss-Seidel Method

Iterative solution of linear systems Ax = b

1. Principle of the method

The Gauss-Seidel method is an **iterative** method for solving a linear system of n equations in n unknowns A x = b. It is an improvement of the Jacobi method: we build a sequence $x^{(0)}$, $x^{(1)}$, $x^{(2)}$, ... that converges to the solution, but each new component is **used immediately** in the computation of the following ones.
In each equation i, we isolate the unknown x_i (this requires $a_{ii} \neq 0$):

$$x_i^{(k+1)} = \left(b_i - \sum_{j<i} a_{ij}\, x_j^{(k+1)} - \sum_{j>i} a_{ij}\, x_j^{(k)} \right) / a_{ii}$$

Key point: for j < i, we use the values $x_j^{(k+1)}$ that have just been computed in the current iteration; for j > i, we use the values of the previous iteration $x_j^{(k)}$. In Jacobi, only the previous iteration is used.

Matrix form

We write A = D + L + U, where D is the diagonal of A, L its strictly lower triangular part and U its strictly upper triangular part. The iteration reads:

$$(D + L)\, x^{(k+1)} = b - U\, x^{(k)} \quad \text{i.e.} \quad x^{(k+1)} = (D + L)^{-1} \left(b - U\, x^{(k)} \right)$$

Convergence condition

The method converges for any initial vector $x^{(0)}$ if, for example, the matrix A is **strictly diagonally dominant** or **symmetric positive definite**:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \text{for every } i$$

More generally, the method converges if and only if the spectral radius of the iteration matrix $G = -(D + L)^{-1} U$ is strictly less than 1.

Stopping criterion

The iterations stop when the difference between two successive iterates is small enough:

$$\max_i \left| x_i^{(k+1)} - x_i^{(k)} \right| < \text{tol} \quad (\text{for example tol} = 10^{-4})$$

2. Worked example

Solve the following system (the same as for Jacobi) with $x^{(0)} = (0, 0, 0)$:

10 x1 + x2 + x3 = 12
2 x1 + 10 x2 + x3 = 13
2 x1 + 2 x2 + 10 x3 = 14

Convergence check: row 1: 10 > 1 + 1 = 2; row 2: 10 > 2 + 1 = 3; row 3: 10 > 2 + 2 = 4. The matrix is strictly diagonally dominant, so Gauss-Seidel converges.

Iteration formulas

$$x1^{(k+1)} = (12 - x2^{(k)} - x3^{(k)}) / 10$$
$$x2^{(k+1)} = (13 - 2\ \mathbf{x1^{(k+1)}} - x3^{(k)}) / 10$$
$$x3^{(k+1)} = (14 - 2\ \mathbf{x1^{(k+1)}} - 2\ \mathbf{x2^{(k+1)}}) / 10$$

The values in bold are the ones that have just been computed in the same iteration.

Iteration 1 (k = 0 to 1)

x1 = (12 - 0 - 0)/10 = 1.2
x2 = (13 - 2×1.2 - 0)/10 = 1.06
x3 = (14 - 2×1.2 - 2×1.06)/10 = 0.948

Iteration 2 (k = 1 to 2)

x1 = (12 - 1.06 - 0.948)/10 = 0.9992
x2 = (13 - 2×0.9992 - 0.948)/10 = 1.00536
x3 = (14 - 2×0.9992 - 2×1.00536)/10 = 0.999088

Table of iterations

k	x1	x2	x3	Max difference
0	0.00000	0.00000	0.00000	-
1	1.20000	1.06000	0.94800	1.20000
2	0.99920	1.00536	0.99909	0.20080
3	0.99956	1.00018	1.00005	0.00518
4	0.99998	1.00000	1.00000	0.00042
5	1.00000	1.00000	1.00000	0.00002

Solution: with tol = 10^{-4}, the method stops at iteration 5. The exact solution is **x = (1, 1, 1)**: substituting into the system confirms it (10+1+1 = 12, 2+10+1 = 13, 2+2+10 = 14). Jacobi needed 9 iterations for the same system.

3. Algorithm

Inputs: matrix A (n×n), vector b, initial vector $x^{(0)}$, tolerance tol, maximum number of iterations N_{max}.
Output: approximation x of the solution of A x = b.

```
GaussSeidel(A, b, x0, tol, Nmax)
  1. n <- size of b
  2. x <- x0
  3. FOR k = 1 TO Nmax DO
  4.     x_old <- copy of x
  5.     FOR i = 1 TO n DO
  6.         s1 <- 0 ; s2 <- 0
  7.         FOR j = 1 TO i-1 DO
  8.             s1 <- s1 + A[i][j] * x[j]       (new values)
  9.         END FOR
 10.         FOR j = i+1 TO n DO
 11.             s2 <- s2 + A[i][j] * x[j]       (old values)
 12.         END FOR
 13.         x[i] <- (b[i] - s1 - s2) / A[i][i]  (x is updated in place)
 14.     END FOR
 15.     IF max |x[i] - x_old[i]| < tol THEN
 16.         RETURN x
 17.     END IF
 18. END FOR
 19. RETURN x      (not converged after Nmax iterations)
```

Remarks

• We need a_{ii} != 0 for every i; otherwise, swap the rows of A.
• Only one vector is needed: x is overwritten component by component (a copy x_old is kept only for the stopping test).
• Cost per iteration: about n^2 operations for a dense matrix, as for Jacobi.
• Gauss-Seidel usually converges faster than Jacobi (here 5 iterations instead of 9), but it is harder to parallelize because each component depends on the previous ones.

Jacobi vs Gauss-Seidel

	Jacobi	Gauss-Seidel
Values used	previous iteration only	newest values available
Vectors stored	x and x_new	x (in place)
Iterations (example)	9	5
Parallelization	easy	difficult